

BÁO CÁO MIDTERM

Implement ls(1)

Môn học: Lập trình hệ thống

Sinh viên: Đặng Tú Nguyên

Lớp: 24SE2

MSSV: 24IT181

Hệ điều hành: NetBSD/amd64

Báo cáo trình bày quá trình xây dựng một chương trình mô phỏng lệnh `ls(1)` trên hệ điều hành NetBSD, sử dụng ngôn ngữ C, tổ chức mã nguồn theo dạng nhiều mô-đun và quản lý phiên bản bằng Git/GitHub.

1. Giới thiệu đề tài

Đề tài yêu cầu xây dựng một phiên bản rút gọn của lệnh UNIX `ls(1)`. Chương trình có nhiệm vụ đọc thông tin tệp và thư mục từ hệ thống tệp, hiển thị danh sách theo nhiều chế độ và hỗ trợ các tùy chọn dòng lệnh tương tự lệnh `ls` trên NetBSD.

Mục tiêu chính của bài làm là vận dụng kiến thức lập trình hệ thống như thao tác với file system, directory, cấu trúc `stat`, `lstat`, `readdir`, xử lý tham số dòng lệnh, cấp phát bộ nhớ động, sắp xếp dữ liệu, định dạng đầu ra và xây dựng chương trình bằng Makefile.

2. Mục tiêu thực hiện

- Xây dựng chương trình `ls` bằng ngôn ngữ C trên NetBSD/amd64.
- Tổ chức chương trình thành nhiều file `.c` và file tiêu đề `.h` để dễ bảo trì và mở rộng.
- Đọc và hiển thị thông tin tệp/thư mục bằng các system call và thư viện chuẩn POSIX.
- Hỗ trợ đầy đủ 20 tùy chọn chính được yêu cầu trong tài liệu đặc tả NetBSD Manual.
- Xử lý linh hoạt nhiều dạng đầu vào như file thường, thư mục, file ẩn, file liên kết tượng trưng (symlink), và duyệt đệ quy thư mục lồng nhau.
- Đảm bảo chương trình biên dịch sạch sẽ với các cờ kiểm tra nghiêm ngặt `-Wall -Wextra -pedantic -std=c99`.
- Sử dụng Git để quản lý phiên bản và lưu trữ mã nguồn trên GitHub theo đúng chuẩn quy định.

3. Các chức năng đã cài đặt

Tùy chọn	Chức năng
-A	Hiển thị tất cả file ẩn, ngoại trừ <code>.</code> và <code>..</code> . Mặc định luôn bật với super-user (root).
-a	Hiển thị tất cả các mục, bao gồm cả file ẩn <code>.</code> và <code>..</code> .
-c	Sử dụng thời gian thay đổi trạng thái tập tin (ctime) cho sắp xếp (-t) hoặc hiển thị (-l). Ghi đề -u.
-d	Hiển thị thư mục như một file thông thường (không duyệt vào trong). Không giải tham chiếu symlink. Ghi đề -R.
-F	Phân loại loại file bằng ký tự chỉ định: <code>/</code> (thư mục), <code>*</code> (thực thi), <code>@</code> (symlink), <code>=</code> (socket), <code> </code> (FIFO), <code>%</code> (whiteout).
-f	Xuất kết quả không sắp xếp (theo đúng thứ tự đọc thực tế của thư mục). Tự động bật kèm -a.
-h	Hiển thị kích thước theo dạng con người đọc được (B, K, M, G, T, P) cho cờ -l và -s. Ghi đề -k.

Tùy chọn	Chức năng
-i	Hiển thị số hiệu inode (file serial number) của từng tệp tin.
-k	Hiển thị kích thước / số block theo đơn vị Kilobyte (1024 bytes). Bị -h ghi đè nếu -h đứng sau.
-l	Hiển thị thông tin chi tiết (long format): chế độ file, số link, owner, group, bytes/device, ngày giờ, đường dẫn. Ghi đè -n.
-n	Hiển thị thông tin chi tiết tương tự -l, nhưng UID/GID được hiển thị dưới dạng số nguyên. Ghi đè -l.
-q	Thay thế các ký tự không in được trong tên file bằng ký tự ? (mặc định khi xuất ra terminal). Ghi đè -w.
-R	Liệt kê cây thư mục theo dạng đệ quy sâu vào tất cả thư mục con gặp phải. Ghi đè -d.
-r	Đảo ngược thứ tự sắp xếp (ngược bảng chữ cái, nhỏ nhất hoặc cũ nhất lên đầu).
-S	Sắp xếp các mục theo kích thước tệp tin (size), mục có kích thước lớn nhất hiển thị trước.
-s	Hiển thị số block hệ thống tệp tin được cấp phát thực tế (đơn vị 512B hoặc BLOCKSIZE). In dòng total khi xuất ra terminal.
-t	Sắp xếp các mục theo thời gian sửa đổi gần nhất trước (mtime, ctime với -c, hoặc atime với -u).
-u	Sử dụng thời gian truy cập gần nhất (atime) cho sắp xếp (-t) hoặc hiển thị (-l). Ghi đè -c.
-w	Hiển thị ký tự không in được ở dạng nguyên bản (raw non-printable). Mặc định khi redirect ra file/pipe. Ghi đè -q.

4. Thiết kế chương trình

Chương trình được chia thành các mô-đun nhằm tách riêng trách nhiệm của từng thành phần:

Tệp / Thành phần	Chức năng
src/ls.h	Khai báo cấu trúc dữ liệu chung (<code>options_t</code> , <code>file_info_t</code> , <code>file_list_t</code>), các enum chế độ và prototype các hàm.
src/compat.h src/compat.c	Lớp tương thích đa nền tảng (POSIX & Win32): giải quyết UID/GID, kiểm tra super-user, device major/minor, lstat, symlink, terminal.
src/options.h src/options.c	Phân tích các tùy chọn dòng lệnh (CLI options parsing) và xử lý đầy đủ các quy tắc ghi đè (precedence & overrides) theo chuẩn NetBSD.
src/file_utils.h src/file_utils.c	Các hàm tiện ích: truy vấn metadata (lstat), định dạng chuỗi quyền 10 ký tự (strmode), định dạng kích thước con người (humanize), thời gian.
src/sort.h src/sort.c	Bộ máy sắp xếp đa tiêu chí sử dụng qsort: sắp xếp theo tên, kích thước, thời gian và hỗ trợ đảo ngược kết quả (-r).
src/display.h src/display.c	Định dạng thông tin hiển thị: canh chỉnh độ rộng các cột động cho đẹp mắt, xuất dạng ngắn (1 entry/dòng), dạng dài (-l), tính tổng số block (total).
src/traverse.h src/traverse.c	Quản lý đối số dòng lệnh: phân loại file thường và thư mục, hiển thị theo thứ tự quy định, duyệt đệ quy thư mục con (-R).
src/main.c	Điểm bắt đầu chương trình: khởi tạo locale, timezone, điều phối luồng thực thi và trả về mã thoát chuẩn (0 khi thành công, >0 khi có lỗi).
Makefile	Tự động biên dịch các file nguồn bằng trình biên dịch cc/gcc, hỗ trợ cả BSD make (NetBSD) và GNU make, hỗ trợ make clean.
.gitignore	Cấu hình loại bỏ file đối tượng (.o) và file thực thi nhị phân (executable) khỏi Git repository.

5. Cấu trúc thư mục

Cấu trúc chính của dự án DangTuNguyen_24IT181_midterm:

```
DangTuNguyen_24IT181_midterm/
├── Makefile           # File cấu hình biên dịch tự động
├── README.md          # Báo cáo chi tiết và tài liệu hướng dẫn
├── .gitignore         # Loại trừ file nhị phân compiled và file rác
└── src/
    ├── compat.c       # Mã nguồn tương thích hệ thống
    ├── compat.h       # Header tương thích hệ thống
    ├── display.c      # Định dạng và canh cột xuất dữ liệu
    ├── display.h      # Header module hiển thị
    ├── file_utils.c   # Truy vấn thuộc tính tệp và định dạng quyền
    ├── file_utils.h   # Header module tiện ích tệp
    ├── ls.h          # Header định nghĩa dữ liệu toàn cục
    ├── main.c         # Hàm main() điều khiển chính
    ├── options.c      # Xử lý cờ dòng lệnh và ưu tiên ghi đè
    ├── options.h      # Header module tùy chọn
    ├── sort.c         # Thuật toán sắp xếp đa tiêu chí
    ├── sort.h         # Header module sắp xếp
    ├── traverse.c     # Duyệt hệ thống thư mục và xử lý đối số
    └── traverse.h     # Header module duyệt thư mục
```

6. Biên dịch chương trình

Dự án sử dụng Makefile để tự động hóa quá trình biên dịch. Các cờ biên dịch nghiêm ngặt được sử dụng:

```
cc -Wall -Wextra -pedantic -std=c99 -O2 -Isrc
```

Để biên dịch toàn bộ chương trình, chạy lệnh make. Lệnh make clean được dùng để xóa file thực thi và các file object (.o).

```
$ cd ~/DangTuNguyen_24IT181_midterm && make clean && make && ./ls
rm -f src/main.o src/options.o src/file_utils.o src/sort.o src/display.o sr
c/traverse.o src/compat.o ls ls.exe
cc -O2 -Isrc -c src/main.c -o src/main.o
cc -O2 -Isrc -c src/options.c -o src/options.o
cc -O2 -Isrc -c src/file_utils.c -o src/file_utils.o
cc -O2 -Isrc -c src/sort.c -o src/sort.o
cc -O2 -Isrc -c src/display.c -o src/display.o
cc -O2 -Isrc -c src/traverse.c -o src/traverse.o
cc -O2 -Isrc -c src/compat.c -o src/compat.o
cc -O2 -Isrc -o ls src/main.o src/options.o src/file_utils.o src/sort.o src/
display.o src/traverse.o src/compat.o
Makefile
README.md
ls
src
$ █
```

Hình 1: Quá trình make clean, make và chạy kiểm tra mặc định ./ls trên máy ảo NetBSD 10.0

7. Kiểm thử chương trình

Chương trình đã được kiểm thử trực tiếp trên máy ảo NetBSD 10.0 (amd64) với đầy đủ các tùy chọn và trường hợp đầu vào theo yêu cầu đề bài:

Lệnh	Mục đích kiểm thử
<code>./ls</code>	Liệt kê file/thư mục thông thường (mặc định 1 entry trên mỗi dòng).
<code>./ls -a</code>	Kiểm tra hiển thị toàn bộ file ẩn kể cả <code>.</code> và <code>...</code>
<code>./ls -A</code>	Kiểm tra hiển thị file ẩn nhưng loại bỏ <code>.</code> và <code>...</code>
<code>./ls -l</code>	Kiểm tra định dạng dài (long listing format), in thông tin chi tiết và dòng total.
<code>./ls -n</code>	Kiểm tra hiển thị thông tin với UID và GID dạng số nguyên (1000, 100).
<code>./ls -lh</code>	Kiểm tra kích thước tệp hiển thị theo dạng con người đọc được (867B, 17K, 31K).
<code>./ls -s / -sk / -sh</code>	Kiểm tra số block cấp phát: chuẩn 512B (-s), đơn vị KB (-sk) và human-readable (-sh).
<code>./ls -F</code>	Kiểm tra phân loại loại file bằng ký tự chỉ định (* cho thực thi, / cho thư mục).
<code>./ls -i</code>	Kiểm tra hiển thị số hiệu Inode thực tế trên hệ thống tệp NetBSD.
<code>./ls -S / -t / -r / -f</code>	Kiểm tra sắp xếp theo kích thước (-S), thời gian (-t), đảo ngược (-r), không sắp xếp (-f).
<code>./ls -lc / -lu / -lcu</code>	Kiểm tra lựa chọn thời gian ctime/atime và quy tắc cờ sau cùng ghi đè.
<code>./ls -d / -R / -Rd</code>	Kiểm tra xem thư mục như file thường (-d), duyệt đệ quy (-R) và quy tắc ghi đè.
<code>./ls Makefile src</code>	Kiểm tra xử lý nhiều đối số (phân loại file thường trước, thư mục sau kèm tiêu đề).
<code>./ls no_such_file ...</code>	Kiểm tra xử lý lỗi khi file không tồn tại: báo lỗi ra stderr và trả về exit code > 0.
<code>BLOCKSIZE=1024 ./ls -s</code>	Kiểm tra biến môi trường BLOCKSIZE điều chỉnh đơn vị tính toán số block.

```
$ echo '=== ./ls -a ===' && ./ls -a && echo '=== ./ls -A ===' && ./ls -A
=== ./ls -a ===
.
..
.gitignore
Makefile
README.md
ls
src
=== ./ls -A ===
.gitignore
Makefile
README.md
ls
src
$
```

Hình 2: Kiểm tra tùy chọn hiển thị file ẩn: cờ -a (bao gồm . và ..) và cờ -A (loại trừ . và ..)

```
$ echo '=== ./ls -l ===' && ./ls -l && echo '=== ./ls -n ===' && ./ls -n
=== ./ls -l ===
total 108
-rw-r--r-- 1 tunguyen users 867 Oct 8 01:27 Makefile
-rw-r--r-- 1 tunguyen users 16907 Oct 8 01:27 README.md
-rwxr-xr-x 1 tunguyen users 31480 Oct 8 01:28 ls
drwxr-xr-x 2 tunguyen users 512 Oct 8 01:28 src
=== ./ls -n ===
total 108
-rw-r--r-- 1 1000 100 867 Oct 8 01:27 Makefile
-rw-r--r-- 1 1000 100 16907 Oct 8 01:27 README.md
-rwxr-xr-x 1 1000 100 31480 Oct 8 01:28 ls
drwxr-xr-x 2 1000 100 512 Oct 8 01:28 src
$
```

Hình 3: Kiểm tra định dạng dài -l (hiển thị tên người dùng tunguyen, nhóm users) và -n (hiển thị UID 1000, GID 100)

```

=== ./ls -lh ===
total 54K
-rw-r--r-- 1 tunguyen users 867B Oct 8 01:27 Makefile
-rw-r--r-- 1 tunguyen users 17K Oct 8 01:27 README.md
-rwxr-xr-x 1 tunguyen users 31K Oct 8 01:28 ls
dwxr-xr-x 2 tunguyen users 512B Oct 8 01:28 src
=== ./ls -s ===
total 108
 4 Makefile
36 README.md
64 ls
 4 src
=== ./ls -sk ===
total 54
 2 Makefile
18 README.md
32 ls
 2 src
=== ./ls -sh ===
total 54K
2.0K Makefile
18K README.md
32K ls
2.0K src
$

```

Hình 4: Kiểm tra dung lượng con người đọc được (-lh), số blocks (-s), kilobyte (-sk) và blocks con người (-sh)

```

$ echo '=== ./ls -F ===' && ./ls -F && echo '=== ./ls -i -F ===' && ./ls -i -F
=== ./ls -F ===
Makefile
README.md
ls*
src/
=== ./ls -i -F ===
2886914 Makefile
2886915 README.md
2886939 ls*
2886913 src/
$

```

Hình 5: Kiểm tra cờ phân loại -F (thêm dấu * cho ls, / cho src) và hiển thị số hiệu Inode với cờ -i

```

&& echo '=== Time -t ===' && ./ls -t && echo '=== Unsorted -f ===' && ./ls -f
=== Sort Size -S ===
ls
README.md
Makefile
src
=== Reverse -r ===
src
ls
README.md
Makefile
=== Time -t ===
ls
src
README.md
Makefile
=== Unsorted -f ===
.
..
src
Makefile
README.md
.gitignore
ls
$

```

Hình 6: Kiểm tra sắp xếp theo kích thước (-S), đảo ngược (-r), theo thời gian (-t) và không sắp xếp (-f)

```

$ echo '=== mtime -l ===' && ./ls -l Makefile && echo '=== ctime -lc ===' && ./ls -lc Makefile && echo '=== atime -lu ===' && ./ls -lu Makefile && echo '=== -lcu (u wins) ===' && ./ls -lcu Makefile
=== mtime -l ===
-rw-r--r-- 1 tunguyen users 867 Oct 8 01:27 Makefile
=== ctime -lc ===
-rw-r--r-- 1 tunguyen users 867 Oct 8 01:27 Makefile
=== atime -lu ===
-rw-r--r-- 1 tunguyen users 867 Oct 8 01:28 Makefile
=== -lcu (u wins) ===
-rw-r--r-- 1 tunguyen users 867 Oct 8 01:28 Makefile
$

```

Hình 7: Kiểm tra thời gian sửa đổi (mtime), thay đổi trạng thái (-lc) và truy cập (-lu), kiểm chứng ghi đè (-lcu)


```
$ echo '=== ./ls -d demo ===' && ./ls -d demo && echo '=== ./ls -R demo ===' &&
./ls -R demo && echo '=== Override -Rd ===' && ./ls -Rd demo
=== ./ls -d demo ===
demo
=== ./ls -R demo ===
file1.txt
sub

demo/sub:
file2.txt
=== Override -Rd ===
demo
$
```

Hình 8: Kiểm tra hiển thị thư mục như file thường (-d), duyệt đệ quy cây thư mục (-R) và ghi đè (-Rd)

```
$ echo '=== Multiple operands ===' && ./ls Makefile demo && echo '=== Error handling ===' && ./ls no_such_file Makefile; echo Exit code: $?
=== Multiple operands ===
Makefile

demo:
file1.txt
sub
=== Error handling ===
ls: no_such_file: No such file or directory
Makefile
Exit code: 1
$
```

Hình 9: Kiểm tra xử lý nhiều đối số (file in trước, thư mục in sau) và xử lý lỗi file không tồn tại (Exit code 1)

```

$ echo '=== Default BLOCKSIZE ===' && ./ls -s Makefile && echo '=== BLOCKSIZE=1024 ===' && ./ls -s Makefile && echo '=== BLOCKSIZE=2048 ===' && ./ls -s Makefile
=== Default BLOCKSIZE ===
4 Makefile
=== BLOCKSIZE=1024 ===
2 Makefile
=== BLOCKSIZE=2048 ===
1 Makefile
$

```

Hình 10: Kiểm tra biến môi trường BLOCKSIZE: mặc định 512B (4 blocks) -> 1024B (2 blocks) -> 2048B (1 block)

8. Quản lý phiên bản bằng Git

Dự án được quản lý nghiêm ngặt bằng Git và đẩy lên GitHub. Các commit được chia nhỏ theo từng giai đoạn phát triển rõ ràng thay vì chỉ commit một lần. Một số commit chính trong lịch sử Git gồm:

- chore: initial project configuration, Makefile and .gitignore – Khởi tạo dự án, thiết lập Makefile và file loại trừ.
- feat(core): implement core data structures and cross-platform POSIX compatibility layer – Xây dựng cấu trúc dữ liệu cốt lõi và lớp tương thích hệ điều hành.
- feat(options): implement NetBSD-compliant CLI options parser with override hierarchy – Hiện thực bộ phân tích cờ dòng lệnh và quy tắc ghi đè NetBSD.
- feat(metadata): implement file metadata extraction, strmode permissions, and sorting engine – Truy vấn metadata tệp, định dạng chuỗi quyền và bộ máy sắp xếp.
- feat(display): implement tabular formatting, recursive directory traversal, and entry point – Hiện thực canh cột đầu ra, duyệt đệ quy và hàm main().
- docs: add comprehensive project report and documentation – Bổ sung tài liệu báo cáo kỹ thuật README.md.
- feat: add real NetBSD screenshots and optimize Makefile/compat for native NetBSD – Tối ưu tương thích NetBSD native và bổ sung ảnh chụp thực tế.

Repository GitHub: https://github.com/tusssstudyit/DangTuNguyen_24IT181_midterm

9. Kết quả đạt được

- Hoàn thành chương trình ls dạng mô-đun bằng ngôn ngữ C chuẩn C99.
- Chương trình biên dịch thành công 100% không cảnh báo (zero warnings) với các cờ -Wall -Wextra -pedantic -std=c99 -O2.
- Hiện thực chính xác toàn bộ 20 tùy chọn của ls(1) theo tài liệu đặc tả NetBSD Manual.
- Có khả năng xử lý hoàn hảo file thường, thư mục, file ẩn, liên kết tượng trưng và duyệt đệ quy không bị lặp vô hạn.
- Xử lý lỗi an toàn, không bị lỗi phân đoạn (Segmentation fault) khi gặp file không tồn tại.
- File Makefile và .gitignore cấu hình chuẩn mực, không commit các file nhị phân rác.
- Toàn bộ mã nguồn đã được đồng bộ lên GitHub theo các quy chuẩn Git chuyên nghiệp.
- Đã thực hiện chạy thực tế trên máy ảo NetBSD 10.0 (amd64) qua tài khoản tunguyen và lưu lại ảnh minh chứng sắc nét.

10. Hạn chế và hướng phát triển

Phiên bản hiện tại đã đáp ứng hoàn chỉnh tập con tất cả các tùy chọn được quy định trong tài liệu NetBSD General Commands Manual được giao. Trong tương lai, chương trình có thể tiếp tục mở rộng thêm các tính năng nâng cao như: định dạng nhiều cột tự động căn chỉnh theo chiều rộng cửa sổ terminal (multi-column - C), hỗ trợ màu sắc phân loại tệp tin theo biến môi trường LSCOLORS, và hỗ trợ các cờ POSIX mở rộng khác.

11. Môi trường phát triển

- **Hệ điều hành thử nghiệm & chạy thực tế:** NetBSD 10.0 (amd64) trên VirtualBox
- **Ngôn ngữ lập trình:** C (Chuẩn C99)
- **Trình biên dịch (Compiler):** cc / gcc (NetBSD native)
- **Công cụ xây dựng (Build tool):** BSD Make & GNU Make
- **Quản lý phiên bản:** Git 2.45+
- **Nền tảng lưu trữ mã nguồn:** GitHub

12. Kết luận

Thông qua đề tài Midterm Project: Implement `ls(1)`, em đã có cơ hội áp dụng trực tiếp các kiến thức nền tảng của môn Lập trình hệ thống và Hệ điều hành vào một bài toán thực tế. Quá trình thực hiện giúp em củng cố sâu sắc kiến thức về kiến trúc file system, directory tree, đọc metadata qua inode và cấu trúc stat, quản lý quyền hạn tệp tin (rwx, Set-UID, Set-GID, Sticky bit), xử lý bộ nhớ động an toàn trong C, tự động hóa biên dịch bằng Makefile và kỹ năng làm việc chuyên nghiệp với Git/GitHub.

13. Tài liệu tham khảo

1. NetBSD 10.1 General Commands Manual: `ls(1)`, `stat(2)`, `lstat(2)`, `opendir(3)`, `readdir(3)`, `closedir(3)`, `readlink(2)`.
2. W. Richard Stevens, Stephen A. Rago. *Advanced Programming in the UNIX Environment (APUE)*, 3rd Edition.
3. IEEE Std 1003.1-2008 / IEEE Std 1003.2 ("POSIX.2") Shell and Utilities Specification.